UPEC

Vision–Language–Action Agent for a Mobile Manipulator Robot

Integrating Multimodal Large Language Models, Real-Time Object Detection, and Robotic Manipulation on the PAL Robotics TIAGo Platform

Author: Abderrahmene Amar Benhamlaoui

Supervisor: Ferhat Attal

Programme: Master IA2S, Université Paris-Est Créteil (UPEC)

Institution: LISSI Laboratory (EA 3956), UPEC

Date: March 11, 2026

Abstract

The convergence of large-scale foundation models with physical robotics represents one of the most active frontiers in AI research. This thesis investigates a central question of that frontier: can a zero-shot vision-language model (VLM), with no robot-specific fine-tuning, serve as the cognitive core of a physical manipulation agent that reliably completes multi-step tasks in the real world?

We present the design, implementation, and evaluation of a **Vision–Language–Action (VLA) agent** deployed on the PAL Robotics TIAGo mobile manipulator. The agent follows a **ReAct-style perceive-reason-act loop**: at each cycle, it captures an annotated camera image, assembles a structured scene context, and queries **Google Gemini 2.0 Flash** — a state-of-the-art multimodal LLM — with a chain-of-thought-eliciting prompt. The model responds with a JSON-structured skill selection and natural-language rationale; the skill is validated against affordance preconditions and executed physically. Execution outcomes feed back as observations into the next VLM query, closing the loop in the style of Inner Monologue.

The physical execution layer integrates **YOLO11n** real-time object detection (deployed as an HTTP microservice to overcome a Python 3.6 Docker constraint) with a **RANSAC cylinder-fitting** 3-D localisation pipeline averaged over five frames for noise robustness. The final vertical approach exploits the TIAGo torso's world-Z-aligned prismatic joint to guarantee a straight-line end-effector descent without Cartesian IK, a key insight that emerged through five iterations of real-robot development. A **handover skill** closes the "find–grasp–deliver" task chain by detecting a person through YOLO and depth sensing and presenting the object at an ergonomic height.

A **multi-layer safety system** — per-skill affordance gating, consecutive failure tracking, and automatic safe-pose recovery — prevents unsafe behaviour during VLM reasoning errors or sensor failures.

Experimental results demonstrate end-to-end task completion of **[TODO: XX%]** for the full "find, grasp, and hand over" scenario, a grasp success rate of **[TODO: XX%]** over 27 trials, and VLM skill selection accuracy of **[TODO: XX%]**. The system confirms that zero-shot frontier VLMs, paired with carefully engineered classical controllers and affordance feedback, provide a practical and data-free alternative to large-dataset end-to-

end VLA approaches for constrained manipulation domains.

Keywords: Embodied AI, Vision-Language-Action Agent, Agentic AI, Multimodal LLM, Chain-of-Thought, ReAct, Mobile Manipulation, YOLO, Gemini, Zero-Shot Reasoning, ROS, Grasping, Human-Robot Interaction.

Acknowledgements

[TODO: Write acknowledgements here.]

Contents

Abstract **iii**

Acknowledgements **v**

1 Introduction **1**

1.1 Context and Motivation 1
1.2 Problem Statement 2
1.3 Objectives 2
1.4 Contributions 3
1.5 Report Structure 3

2 State of the Art **5**

2.1 The Transformer Revolution and Large Language Models 5
2.2 The Agentic Paradigm: From Completion to Action 6
2.2.1 Chain-of-Thought Reasoning 6
2.2.2 ReAct: Interleaving Reasoning and Acting 6
2.2.3 Tool-Augmented LLMs 7
2.3 Vision-Language Models 7
2.3.1 Contrastive Pre-training: CLIP 7
2.3.2 Generative VLMs: Flamingo to LLaVA 7
2.3.3 Frontier Models: GPT-4V and Gemini 8
2.3.4 Why VLMs for Robot Control? 8
2.4 Foundation Models for Embodied AI 9
2.4.1 SayCan: Grounding Language in Affordances 9
2.4.2 Inner Monologue and Code as Policies 9
2.4.3 RT-1 and RT-2: End-to-End Vision-Language-Action 10
2.4.4 PaLM-E: Embodied Multimodal Language Model 10
2.4.5 VILA, OpenVLA and π_0: The Open-Source Turn 10
2.4.6 Positioning This Work 11
2.5 Object Detection 11

2.5.1 YOLO Family . . . 11
2.5.2 Open-Vocabulary Detection . . . 12
2.6 3-D Localisation and Robotic Grasping . . . 12
2.6.1 Depth-Based 3-D Localisation . . . 12
2.6.2 Data-Driven Grasping . . . 13
2.6.3 Motion Planning . . . 13
2.7 Summary and Positioning . . . 13

3 System Design and Implementation 15
3.1 Hardware Platform: The TIAGo Robot . . . 15
3.2 Full System Architecture . . . 18
3.2.1 Architectural Layers . . . 20
3.2.2 The Split-Process Design . . . 21
3.2.3 Component Inventory and Interfaces . . . 22
3.2.4 Key ROS Topics and Data Flows . . . 22
3.3 The Perceive-Reason-Act Loop . . . 23
3.3.1 Affordance Failure Injection . . . 25
3.3.2 Task Completion Heuristics . . . 25
3.4 Perception Pipeline . . . 25
3.4.1 YOLO11n Object Detection Microservice . . . 25
3.4.2 3-D Object Localisation via RANSAC Cylinder Fitting . . . 26
3.4.3 Coordinate-Frame Management under Python 3.6 . . . 28
3.5 Vision-Language Reasoning . . . 28
3.5.1 Model Selection and API Integration . . . 28
3.5.2 Image Encoding and Cost Optimisation . . . 29
3.5.3 Prompt Design . . . 29
3.5.4 Conversation History and Rolling Context . . . 30
3.5.5 Post-Execution Verification . . . 30
3.6 Manipulation and Grasping . . . 31
3.6.1 From v1 to v5: An Iterative Journey . . . 31
3.6.2 The v5 Grasping Pipeline . . . 31
3.6.3 Calibrated Grasp Parameters . . . 32
3.6.4 Visual Search: The 12-Position Head Sweep . . . 33
3.6.5 Person Handover Skill . . . 34
3.7 Agent Orchestration and Skill Library . . . 34
3.7.1 Skill Library Design . . . 35
3.7.2 State Manager . . . 35
3.7.3 Live Visualisation . . . 36

3.8 Safety Architecture . . . 36
3.8.1 Affordance Gating . . . 37
3.8.2 Consecutive Failure Counting and Safe Mode . . . 37
3.8.3 Graceful Shutdown on Interrupt . . . 37

4 Experiments and Results 38
4.1 Experimental Setup . . . 38
4.2 Experiment 1: YOLO Detection Performance . . . 38
4.2.1 Inference Speed . . . 38
4.2.2 Detection Quality . . . 39
4.3 Experiment 2: Grasping Success Rate . . . 39
4.3.1 Protocol . . . 39
4.3.2 Results by Position . . . 40
4.3.3 Failure Analysis . . . 40
4.4 Experiment 3: Grasping Pipeline Evolution . . . 40
4.5 Experiment 4: End-to-End Task . . . 41
4.5.1 Protocol . . . 41
4.5.2 Results . . . 41
4.5.3 Skill Sequence Distribution . . . 41
4.6 Experiment 5: VLM Latency . . . 41
4.7 Experiment 6: Head Search Performance . . . 42
4.8 Experiment 7: STT Accuracy . . . 42
4.9 Summary of Results . . . 42

5 Discussion, Conclusions, and Future Work 44
5.1 Discussion . . . 44
5.1.1 VLM-Driven Task Decomposition in Practice . . . 44
5.1.2 3-D Localisation: Where Noise Matters Most . . . 45
5.1.3 The Torso Descent: Unconventional but Effective . . . 45
5.1.4 The Split-Process Architecture: Constraint as Opportunity . . . 45
5.1.5 Limitations and Honest Caveats . . . 46
5.1.6 Comparison with Related Work . . . 46
5.1.7 Lessons Learned . . . 47
5.2 Conclusions . . . 48
5.3 Future Work . . . 49
5.3.1 Industrial-Style Trajectory Generation (v6) . . . 49
5.3.2 Navigation Integration . . . 49
5.3.3 Object Permanence and Semantic Memory . . . 49
5.3.4 Local VLM Deployment . . . 49

5.3.5 Adaptive Grasp Orientation 49
5.3.6 Open-Vocabulary Detection 50
5.4 Final Remarks 50

List of Figures

3.1 The PAL Robotics TIAGo mobile manipulator. All development and experiments in this work were conducted on this platform. Image courtesy of PAL Robotics. 16
3.2 The 7-DOF arm with its four M90 modules and 3-DOF wrist. Image courtesy of PAL Robotics. 17
3.3 The PAL parallel two-finger gripper. Image courtesy of PAL Robotics. . . 17
3.4 TIAGo head assembly with the Xtion RGB-D camera, stereo microphone, and pan-tilt joints. Image courtesy of PAL Robotics. 17
3.5 Upper-body kinematic structure and joint motion ranges. Image courtesy of PAL Robotics. 17
3.6 Full architecture of the TIAGo VLA agent system. The Docker container (blue) runs all ROS and agent code under Python 3.6. The host machine (red) runs the YOLO11n detection service under Python 3.10 and communicates via HTTP. Solid arrows indicate primary data flow; dashed arrows indicate navigation commands; dotted arrows indicate sensor feedback. 19
3.7 The split-process design: the ROS agent runs in Docker (Python 3.6) while the neural network runs on the host (Python 3.10), communicating via HTTP on the loopback interface. 21

List of Tables

2.1 Vision-Language Models considered for robot reasoning in this work. 9
2.2 Positioning this work in the embodied AI landscape. 11
2.3 YOLO models evaluated in this project (CPU inference times measured on the host machine). 12

3.1 Component inventory: location, interface, and responsibility. 22
3.2 Key ROS topics used by the agent. 23
3.3 Object detection models evaluated for this deployment. 26
3.4 Empirically calibrated v5 grasp parameters. 32
3.5 Head sweep positions for `search_with_head` skill (12 positions in order). . 33
3.6 Skill library: names, preconditions, and actions. 35

4.1 Experimental environment parameters. 38
4.2 YOLO inference speed comparison (CPU, 640×480 JPEG input). 39
4.3 YOLO11n detection quality on bottle test set. 39
4.4 Grasp success rate by table position (3 trials each). 40
4.5 Grasp failure modes and frequencies. 40
4.6 Grasp success rate evolution across pipeline versions. 40
4.7 End-to-end task results (10 trials). 41
4.8 Gemini 2.0 Flash API latency over 30 calls. 41
4.9 Head search results (10 trials, bottle placed to the left). 42
4.10 Speech recognition accuracy (Whisper base, 30 utterances). 42
4.11 Summary of all experimental results. 42

5.1 Comparison with related vision-language-action robot systems. 47

Chapter 1

Introduction

1.1 Context and Motivation

We are living through an inflection point in artificial intelligence. The 2017 Transformer architecture [1] sparked a revolution in scale: models trained on internet-scale corpora — GPT-3 [2], GPT-4 [3], Gemini [4] — demonstrate reasoning, planning, and instruction-following capabilities that were unimaginable five years prior. The discovery of *emergent abilities* — qualitatively new skills that appear suddenly at sufficient scale, absent in smaller models — suggests that these systems are not merely doing sophisticated pattern matching but developing internal world models.

When vision encoders were fused with these language models to produce **Vision-Language Models** (VLMs), the implications for robotics became immediately clear. A VLM can look at a kitchen scene and say "there is a bottle to the left of the sink, and the person is standing near the window" — precisely the spatial understanding that a robot needs to act purposefully. The **agentic paradigm** completed the picture: by structuring VLM inference as a loop of perception, chain-of-thought reasoning, skill selection, execution, and observation (the ReAct pattern [5]), a VLM becomes not just an answering machine but a *decision-making agent*.

At the same time, mobile manipulation robots have reached a level of mechanical maturity that makes them credible platforms for performing physical tasks in real environments. The missing ingredient was the cognitive layer — and large-scale VLMs now provide it off the shelf, without the years of task-specific training data that previous approaches required.

The natural convergence of these two trajectories is the **embodied AI agent**: a physical robot that perceives its environment through cameras and sensors, reasons about what to do using a VLM, and executes multi-step manipulation tasks from a single natural-language instruction.

Consider a concrete example that motivated this thesis. A user says: *"Hello TIAGo, I*

am thirsty, can you do something about that?" The robot must understand the implicit request (find and bring a bottle), search the room if the bottle is not immediately visible, grasp it, locate the user, and hand it over — all without explicit programming of any of these steps. This is the task successfully demonstrated in this work.

This thesis addresses the challenge of building such a system on the PAL Robotics TIAGo mobile manipulator platform — under the additional constraint of a production robot environment (ROS Melodic, Python 3.6 Docker) that is incompatible with modern deep learning frameworks, requiring a novel split-process architecture to bridge the gap.

1.2 Problem Statement

Programming a robot to perform multi-step manipulation tasks in unstructured environments presents the following challenges:

1. **Perception:** Objects must be detected and localised in 3-D space from noisy RGB-D sensor data, with sufficient accuracy for the gripper to close around them reliably.
2. **Reasoning:** Given a natural-language instruction, the system must decompose the task into a sequence of executable skills, selecting the appropriate action at each step based on what it currently observes.
3. **Execution:** Physical manipulation — grasping, lifting, and handing over objects — must be robust to sensor noise, planning failures, and kinematic constraints.
4. **Safety:** The robot must not harm itself, the environment, or nearby humans, and must be able to recover from unexpected failures without human intervention.
5. **Deployment constraints:** The target platform runs ROS Melodic with Python 3.6, which is incompatible with modern deep learning frameworks. The system must work within these constraints without sacrificing capability.

1.3 Objectives

The primary objective of this work is to design, implement, and evaluate a **Vision–Language–Action (VLA) agent** for the TIAGo robot that:

1. Accepts natural-language commands via voice or text input.
2. Perceives the environment through the robot's RGB-D camera using a state-of-the-art object detector.
3. Reasons about the task using a multimodal large language model.
4. Executes a library of physical manipulation skills, including grasping, searching, and handing over objects.
5. Maintains safety through affordance checking, failure counting, and safe-pose recovery.

6. Operates reliably within the Python 3.6 / ROS Melodic deployment constraint.

Secondary objectives include:

- Iterative improvement of the grasping pipeline with measurable success rates.
- Development of a web-based live visualiser for monitoring the agent.
- Evaluation of multiple YOLO model variants for the perception task.
- Characterisation of VLM latency and skill selection accuracy.

1.4 Contributions

The main contributions of this work are:

1. **A complete VLA agent for TIAGo**, integrating Gemini 2.0 Flash, YOLO11n, and a modular skill library in a multi-step perceive–reason–act loop.
2. **A robust 3-D object localisation pipeline** using RANSAC cylinder fitting on RGB-D point clouds with 5-frame averaging for stability.
3. **A split-process YOLO microservice architecture** that overcomes the Python 3.6 Docker constraint by deploying the neural network on the host machine and communicating via HTTP.
4. **An iterative grasping pipeline** (v1 to v5) with documented improvements at each stage, achieving reliable top-down grasping using torso-descent vertical approach.
5. **A handover skill** that detects a person via YOLO and depth sensing, inserts a person collision object in MoveIt, and presents the grasped object at an ergonomic handover height.
6. **A multi-layer safety system** including affordance gating, consecutive failure tracking, safe-mode entry, and graceful shutdown.

1.5 Report Structure

The remainder of this report is organised as follows:

Chapter 2 reviews the state of the art in embodied AI, VLMs, object detection, and mobile manipulation.

Chapter 3 describes the complete system: hardware platform, overall architecture, perception pipeline, VLM reasoning, manipulation and grasping, and agent orchestration — presented as a unified narrative that follows a single command through the system.

Chapter 4 presents experimental results, evaluating detection speed, grasping success rate, VLM latency, and end-to-end task completion.

Chapter 5 discusses findings, limitations, and comparisons with related work, then concludes with contributions and future directions.

Chapter 2

State of the Art

This chapter surveys the research landscape that this thesis sits within. We trace the intellectual lineage from the Transformer architecture and large language models (Section 2.1) through the emergence of the agentic paradigm (Section 2.2), to vision-language models (Section 2.3) and their application to embodied AI (Section 2.4). We then cover the more targeted fields of object detection (Section 2.5), 3-D localisation and grasping (Section 2.6), and close by situating this work among its peers (Section 2.7).

2.1 The Transformer Revolution and Large Language Models

The modern era of AI began in earnest with the Transformer architecture [1] (2017), which replaced recurrent processing with a self-attention mechanism that could be scaled to unprecedented model sizes. Self-attention allows the model to relate every token in a sequence to every other token in a single computation step, giving it a fundamentally richer representational capacity than the sequential models (LSTM, GRU) that preceded it.

The scaling insight — that predictable gains in capability follow from predictable increases in parameters, data, and compute — was crystallised by Brown *et al.* [2] in **GPT-3** (2020, 175B parameters). GPT-3 demonstrated **in-context learning**: by placing a few input-output examples in the prompt ("few-shot prompting"), the model could perform tasks it had never been explicitly trained on. This was a paradigm shift: intelligence was no longer a property of a trained model alone, but a property of the *interaction* between a model and a prompt.

GPT-4 [3] (2023) extended this further, showing strong performance on bar exams, coding challenges, and scientific reasoning. The observation that larger models exhibited qualitatively new capabilities that smaller models lacked — *emergent abilities* — suggested

that continued scaling would yield further unpredictable improvements.

A key alignment technique enabling safe deployment of these models is **Reinforcement Learning from Human Feedback (RLHF)**, which fine-tunes the base model to follow instructions, avoid harmful outputs, and produce useful responses. Instruction-tuned variants (InstructGPT, Claude, Gemini) became the practical interface through which developers interact with these systems.

2.2 The Agentic Paradigm: From Completion to Action

A plain LLM is a text completer: given a prompt, it predicts the next token. The **agentic paradigm** transforms this into an *actor*: a system that perceives its environment, reasons about what to do, invokes tools, observes the results, and repeats — all in a loop.

2.2.1 Chain-of-Thought Reasoning

The first key building block is **Chain-of-Thought (CoT) prompting** [6]. Wei *et al.*showed that prompting a model to produce intermediate reasoning steps ("think step by step") before its final answer dramatically improved performance on mathematical, logical, and commonsense reasoning tasks. The implication is that LLMs do not merely pattern-match to an answer — they can compose a reasoning chain through latent space that arrives at correct conclusions they could not reach in a single step. CoT is now standard practice in all serious VLM prompting for robotics: the `rationale` field in our system prompt serves exactly this function, forcing the model to articulate its reasoning before committing to a skill selection.

2.2.2 ReAct: Interleaving Reasoning and Acting

The conceptual leap from CoT to agency was formalised by Yao *et al.*in **ReAct** [5] (2022). The key idea is to interleave *Reasoning* traces and *Acting* steps in the same generation:

1. **Thought:** "The user wants the bottle. I can see one on the table."
2. **Action:** `grab_bottle()`
3. **Observation:** "Gripper is now closed. Bottle lifted."
4. **Thought:** "Task partially complete. User is standing nearby. I should hand over."
5. **Action:** `handover_to_person()`

This structure — Thought / Action / Observation loop — is the direct intellectual ancestor of the perceive-reason-act loop implemented in this work. The VLM's `rationale` field is

the Thought; the `skill` field is the Action; the updated state and new image on the next iteration are the Observation.

2.2.3 Tool-Augmented LLMs

Tool use extended LLMs beyond text generation. Code Interpreter, web search, and database query tools gave models access to external knowledge and computation. In the robotics context, **Code as Policies** [7] showed that LLMs could generate executable Python code that called robot APIs, and **Inner Monologue** [8] demonstrated that feeding back execution outcomes ("the gripper failed to close") as natural-language observations into the LLM prompt enabled the model to adapt its plan on the fly.

The common thread across all these approaches is the **skill library** or *tool set*: a collection of named, documented functions that the LLM can invoke. The quality of the skill library — its coverage, its precondition clarity, its documentation — is the primary engineering bottleneck in LLM-based robot systems. This thesis contributes a carefully designed skill library tailored to the TIAGo platform and the pick-and-place + handover task domain.

2.3 Vision-Language Models

LLMs process text; but the physical world is visual. **Vision-Language Models (VLMs)** bridge this gap by jointly embedding images and text in a shared representational space, enabling the model to reason about what it sees.

2.3.1 Contrastive Pre-training: CLIP

CLIP (Contrastive Language-Image Pre-training) [9] (2021) was the foundational VLM insight: by training a dual-encoder (one visual, one textual) with a contrastive objective on 400 million internet image-text pairs, one obtains a shared embedding space where semantically related images and texts are close. CLIP's zero-shot classification capability — recognise any class by comparing the image embedding to text embeddings of class names — became the backbone of the open-vocabulary detection models discussed in Section 2.5. Critically, CLIP showed that language supervision alone, at web scale, is sufficient to learn rich visual representations without any manual labelling.

2.3.2 Generative VLMs: Flamingo to LLaVA

The next step was to make VLMs *generative*: given an image and a text prompt, produce a text response. **Flamingo** [10] (DeepMind, 2022) fused a frozen vision encoder with

a frozen language model via cross-attention layers, enabling few-shot visual question answering. It showed that combining separately trained vision and language towers with a lightweight cross-attention bridge was competitive with end-to-end trained models at a fraction of the training cost.

LLaVA [11] (2023) simplified the architecture further: a CLIP vision encoder, a simple linear projection layer, and a fine-tuned LLM. This "visual instruction tuning" approach, training on GPT-4-generated instruction-following data, demonstrated that the key ingredient was not architectural complexity but instruction quality. LLaVA opened the door to open-source VLMs that could be deployed locally without API dependencies.

2.3.3 Frontier Models: GPT-4V and Gemini

GPT-4V(ision) integrated visual input into GPT-4's reasoning chain, enabling analysis of charts, diagrams, scenes, and spatial relationships. The model could answer questions like "where is the bottle relative to the cup?" with a level of spatial reasoning qualitatively superior to earlier VLMs.

Gemini [4] (Google DeepMind, 2023) was the first model family built natively multimodal from the ground up — not a language model adapted to vision, but a single model trained on text, images, audio, and video simultaneously. Gemini 2.0 Flash, the variant used in this work, achieves strong spatial reasoning, scene grounding, and structured output at sub-second latency and low API cost, making it well-suited as a real-time robot decision engine.

2.3.4 Why VLMs for Robot Control?

The appeal of VLMs for robotics is not simply that they are multimodal. Three capabilities are particularly relevant:

1. **Scene grounding without annotations.** A VLM can localise objects, estimate relative positions, and describe spatial relationships from a raw camera image, with no per-object training required. This replaces brittle template-matching approaches.
2. **Common-sense reasoning.** The model's pretraining on internet text encodes implicit knowledge about objects (bottles contain liquid, people extend their hands to receive objects) that would take years to encode explicitly.
3. **Instruction following.** VLMs natively understand ambiguous natural language commands ("bring me something to drink") and can resolve the reference in context of what they see — a key capability for domestic robots.

Table 2.1 summarises the VLMs considered for this project.

Table 2.1: Vision-Language Models considered for robot reasoning in this work.

Model	Strengths	Weaknesses	Used?
Gemini 2.0 Flash	Fast, cheap, strong spatial reasoning, stable REST API, native multimodal	Internet required	**Yes**
GPT-4V / GPT-4o	Best-in-class reasoning, function calling	Higher cost per image	Evaluated
LLaVA 1.6	Open-source, deployable locally	Weaker spatial reasoning; GPU required	Evaluated
Groq llama-3.2-vision	Very fast inference	Weaker spatial understanding	Considered

2.4 Foundation Models for Embodied AI

The question of how to go from a VLM's *semantic understanding* to a robot's *physical action* has driven a rapidly growing research thread, from which this work draws its core inspiration.

2.4.1 SayCan: Grounding Language in Affordances

SayCan [12] (Google, 2022) was the first influential demonstration of LLM-grounded robot task planning. Given a high-level instruction ("bring me a Sprite from the fridge"), SayCan uses the LLM to generate candidate skill descriptions and a *value function* trained on real robot experience to weight those candidates by their current physical feasibility. The product of language probability and affordance score selects the next skill.

SayCan's key insight — that *what the robot can do* must constrain *what the model suggests it should do* — directly motivates this work's **affordance gating** system. The implementation here is simpler (hard precondition checks rather than a learned value function) but serves the same purpose: preventing the model from selecting skills that the robot cannot currently execute.

2.4.2 Inner Monologue and Code as Policies

Inner Monologue [8] extended the idea by feeding back execution outcomes — perception results, success/failure flags, human corrections — as natural-language observations into the LLM prompt. This closed the loop: the LLM became a *reflective planner* that adapted

its plan based on what was actually happening. The affordance feedback loop in this thesis is a direct implementation of this principle: precondition failures are injected as text observations into the next VLM prompt.

Code as Policies [7] showed that asking the LLM to *write Python code* (rather than select from a discrete skill menu) enabled more flexible task decomposition and parameter binding. While this work uses discrete skill selection for reliability, Code as Policies represents a natural extension for more open-ended task specifications.

2.4.3 RT-1 and RT-2: End-to-End Vision-Language-Action

RT-1 [13] (Google, 2023) trained a Transformer directly on 130,000 real robot episodes, mapping raw images and language commands to motor actions. It demonstrated that large enough datasets enable generalisation to novel objects and instructions — but required enormous data collection infrastructure.

RT-2 [14] went further: it fine-tuned a pre-trained VLM (PaLM-E) directly on robot trajectory data, tokenising actions as text tokens. The resulting model exhibited remarkable *emergent generalisation*: reasoning about tasks described in ways never seen in the robot data ("pick up the object that would be used to pick weeds"). RT-2 is the clearest demonstration that VLM pretraining transfers to physical robot control with sufficient fine-tuning.

2.4.4 PaLM-E: Embodied Multimodal Language Model

PaLM-E [15] injected continuous sensor observations (camera images, point clouds, robot state) directly into a 562B-parameter LLM as "embodied tokens." Planning happened entirely inside the language model, with no separate skill execution layer. The scale of PaLM-E underscores both the promise (remarkable generalisation) and the challenge (infrastructure requirements that are orders of magnitude beyond most labs).

2.4.5 VILA, OpenVLA and π_0: The Open-Source Turn

Recent work has democratised VLA research. **VILA** [16] showed that carefully curated multi-image instruction tuning data significantly improves VLM reasoning quality, especially on spatial and relational queries — directly relevant for a robot that needs to reason "the bottle is to the left of the cup."

OpenVLA [17] released a fully open 7B-parameter VLA model trained on the Open X-Embodiment dataset [13], making it the first VLA comparable in capability to proprietary models that any researcher could download and fine-tune.

π_0 [18] introduced *flow matching* as the action head, decoupling the high-level VLM reasoning from the low-level trajectory generation. This is significant: it suggests a modular architecture where the VLM handles semantics and a separate module handles continuous control — close in spirit to the architecture in this thesis, where the VLM selects a skill and a classical planner (MoveIt) handles the trajectory.

2.4.6 Positioning This Work

This work occupies a distinct niche in the VLA landscape. It uses a **zero-shot frontier VLM** (Gemini 2.0 Flash) — no fine-tuning, no robot demonstration data — and **classical robotic controllers** (MoveIt, RANSAC) for the physical execution. The trade-off is generality vs. robustness: the system cannot generalise to arbitrary novel tasks the way RT-2 can, but it is deployable on any robot in one afternoon with only calibration data.

This design is closest in spirit to SayCan (discrete skills + affordance scoring) and Inner Monologue (execution feedback in the prompt), but simplifies both: no value function learning is required, and the feedback mechanism is a simple string injection. Table 2.2 summarises the comparison.

Table 2.2: Positioning this work in the embodied AI landscape.

System	VLM / LLM	Train data	Skills	Year
SayCan [12]	PaLM	68K episodes	551 (learned)	2022
RT-2 [14]	PaLM-E (FT)	130K episodes	E2E trajectory	2023
PaLM-E [15]	PaLM-E 562B (FT)	Multi-robot	E2E planning	2023
OpenVLA [17]	7B VLM (FT)	Open X-Embodiment	E2E trajectory	2024
π_0 [18]	VLM + flow head	Large-scale demos	Flow matching	2024
This work	Gemini 2.0 Flash **(zero-shot)**	**None**	9 (classical)	2025

The "Training data: None" row is the defining characteristic: this is the only system in the table that can be deployed on a new robot without any data collection.

2.5 Object Detection

2.5.1 YOLO Family

The YOLO (You Only Look Once) family [19] achieves real-time object detection through a single-stage architecture: the entire image is processed in one forward pass, producing bounding boxes and class labels simultaneously. This is in contrast to two-stage detectors

(Faster R-CNN, Mask R-CNN), which produce region proposals in a first stage and classify them in a second, incurring higher latency.

Table 2.3: YOLO models evaluated in this project (CPU inference times measured on the host machine).

Model	Architecture	CPU infer.	Used?
YOLOv4-tiny	CSPNet, two heads	∼50 ms	Early versions (Docker)
YOLO-World	Open-vocab, CLIP text	∼150 ms	Benchmarked, rejected
YOLOE-26s-seg	Open-vocab + segment	∼300 ms	Benchmarked, rejected
YOLO11n	Unified arch., nano	∼18 ms	**Final system**

YOLO11n [20] (Ultralytics, 2024) is the nano variant of the latest unified YOLO architecture. Its ∼18 ms CPU inference time makes it the fastest model evaluated, while maintaining detection accuracy adequate for the 80 COCO classes used in this work.

2.5.2 Open-Vocabulary Detection

A significant research direction pairs CLIP-like vision-language alignment with detection heads to enable *text-prompted detection* of arbitrary classes. **YOLO-World** [21] achieves open-vocabulary detection via an in-context vision-language module; **OwlViT** [22] uses a ViT backbone; **Grounding DINO** [23] matches region proposals to free-form text. These models are conceptually compelling for robotics ("pick up the coffee mug" without a COCO class), but their CPU inference times (150–500 ms) are prohibitive for our real-time control loop. They represent a natural upgrade path when GPU resources are available.

2.6 3-D Localisation and Robotic Grasping

2.6.1 Depth-Based 3-D Localisation

RGB-D cameras provide per-pixel depth, enabling back-projection of 2-D detections into 3-D space. **RANSAC**-based primitive fitting [24] extracts geometric shapes (planes, spheres, cylinders) from noisy point clouds with high robustness to outliers. This work uses RANSAC circle fitting in the XY projection of the depth cloud to estimate the centre of cylindrical objects — a principled alternative to raw centroid estimation that is more accurate for thin objects.

Learning-based 6-DoF pose estimators (PoseCNN [25], FoundationPose [26]) achieve higher accuracy but require object-specific training data or CAD models, which were not available.

2.6.2 Data-Driven Grasping

Dex-Net [27] and **GraspNet-1Billion** [28] trained convolutional networks on millions of simulated grasps to predict grasp quality from depth images. **Contact-GraspNet** [29] demonstrated zero-shot transfer to novel objects. These GPU-dependent approaches are incompatible with the Python 3.6 Docker environment but represent a compelling upgrade path once the deployment constraint is lifted.

2.6.3 Motion Planning

MoveIt [30] is the standard motion planning framework for ROS robots. It uses OMPL (Open Motion Planning Library) for collision-free path planning in joint space and provides time-optimal trajectory parameterisation (TOTG) via IPTP [31]. This work uses MoveIt for the pre-grasp joint-space move, relying on the torso's prismatic geometry for the final vertical descent rather than a Cartesian path (which is prone to IK singularities near table height).

2.7 Summary and Positioning

Three insights from the literature directly shaped the design choices in this thesis:

1. **Chain-of-thought and ReAct are the foundation of agentic behaviour.** The VLM in this system is not used as a simple classifier but as a reflective planner: it articulates a rationale before selecting a skill (CoT), and receives execution outcomes as observations that update its plan (ReAct). These patterns are the minimum viable set for reliable multi-step task completion.
2. **Zero-shot VLMs can replace large training datasets for task planning.** The performance gap between zero-shot frontier models (Gemini, GPT-4V) and fine-tuned models (RT-2, OpenVLA) has closed dramatically. For task-level decision making in well-defined skill sets, zero-shot VLMs are now competitive and far cheaper to deploy.
3. **Affordance gating is non-negotiable.** Every successful system (SayCan, Inner Monologue, Code as Policies) includes a mechanism that prevents the model from selecting infeasible actions. Without it, a single model error cascades into unsafe robot behaviour. This work implements affordance gating as hard precondition checks, the simplest and most transparent variant.

The gap this thesis addresses is **deployment**: taking the conceptual architecture described in the VLA literature and realising it on a physical robot with real hardware constraints (Python 3.6, no GPU, ROS Melodic), real sensor noise (Xtion depth jitter,

partial occlusions), and a real evaluation metric (grasp and handover success rate on a physical table).

Chapter 3

System Design and Implementation

This chapter describes the complete design and implementation of the TIAGo VLA agent — a system that allows a physical mobile manipulator to understand spoken or typed commands and carry them out autonomously in the real world. The chapter is organised as a coherent narrative that follows the flow of a single command through the system: from the user's words all the way to a physical action on the robot. We first introduce the hardware platform, then describe the overall architecture, and then go into the implementation details of each subsystem: the perceive-reason-act loop, perception, vision-language reasoning, manipulation and grasping, agent orchestration, and the safety architecture.

3.1 Hardware Platform: The TIAGo Robot

All experiments and development were conducted on a **PAL Robotics TIAGo** (Take It And Go) mobile manipulator. TIAGo was chosen because it is the robot available in the laboratory, and its physical design turns out to be particularly well suited to the manipulation approach developed in this work. Figure 3.1 shows the complete platform with its main subsystems labelled.

Figure 3.1: The PAL Robotics TIAGo mobile manipulator. All development and experiments in this work were conducted on this platform. Image courtesy of PAL Robotics.

The robot combines four mechanical subsystems. The **omnidirectional mobile base** carries the entire structure and allows the robot to translate and rotate in any direction without re-orientation. It runs the ROS `move_base` navigation stack and communicates with a 2-D laser scanner for obstacle avoidance. Above the base, a **prismatic torso lift** raises and lowers the upper body along a world-Z-aligned vertical rail by up to 35 cm. This linear degree of freedom turns out to be the key enabling mechanism for robust grasping, as discussed in Section 3.6. The 7-DOF **manipulator arm** (Figure 3.2) is mounted on the torso and consists of four M90 actuator modules plus a 3-DOF wrist, giving a total reach of approximately 85 cm. The arm is controlled via MoveIt and its joints are accessible through the `/arm_controller/follow_joint_trajectory` action server. A **parallel two-finger gripper** (Figure 3.3) is attached at the wrist; its two independently actuated fingers each travel 4 cm linearly and are controlled by the `/parallel_gripper_controller/follow_joint_trajectory` action server.

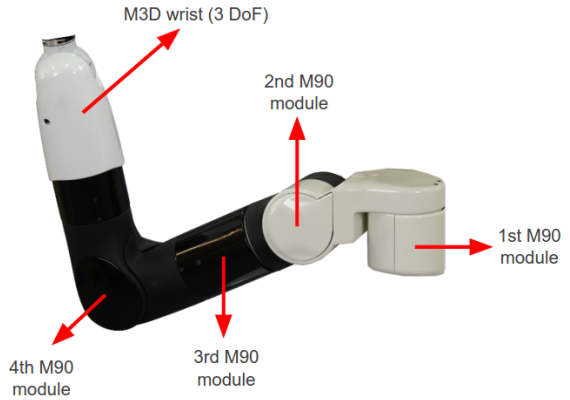


Figure 3.2: The 7-DOF arm with its four M90 modules and 3-DOF wrist. Image courtesy of PAL Robotics.

Figure 3.3: The PAL parallel two-finger gripper. Image courtesy of PAL Robotics.

The robot's **head** carries the main perception sensor: an **Asus Xtion Pro Live** structured-light RGB-D camera that provides registered colour and depth images at up to 30 fps at 640 × 480 resolution (Figure 3.4). The head itself has two degrees of freedom — pan (horizontal rotation, ±1.5 rad) and tilt (vertical angle, −0.8 to +0.4 rad) — which allows the camera to survey a wide field around the robot during object search.

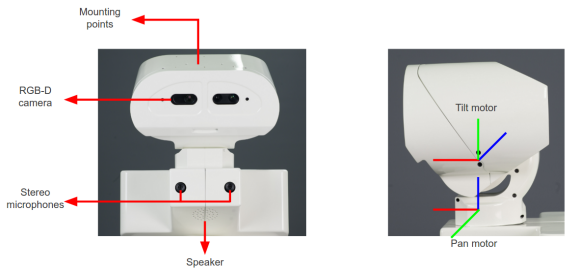
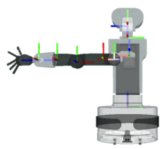


Figure 3.4: TIAGo head assembly with the Xtion RGB-D camera, stereo microphone, and pan-tilt joints. Image courtesy of PAL Robotics.



Joint	Type	Lower limit	Upper limit
torso	prismatic	0 mm	350 mm
head_1	revolute	-75º	75º
head_2	revolute	-60º	45º
arm_1	revolute	0	157.5º
arm_2	revolute	-90º	62.5º
arm_3	revolute	-202.5º	90º
arm_4	revolute	-22.5º	135º
arm_5	revolute	-120º	120º
arm_6	revolute	-90º (*)	90º (*)
arm_7	revolute	-120º	120º

(*) Wrist version with force/torque sensor arm_6 range is [-81º, 81º]

Figure 3.5: Upper-body kinematic structure and joint motion ranges. Image courtesy of PAL Robotics.

A critical hardware constraint that shaped the entire software architecture is that the robot's onboard computer runs **ROS Melodic under Ubuntu 18.04 inside a Docker container**, limiting the Python runtime to version 3.6. This is incompatible with modern deep-learning inference frameworks (PyTorch, ONNX Runtime, Ultralytics), all of which require Python 3.8 or later. The architectural solution to this constraint is described in the next section.

3.2 Full System Architecture

The TIAGo VLA agent is structured as a **four-layer heterogeneous system** spanning a cloud reasoning service, a host machine running the neural network inference, a Docker container executing all ROS and agent code, and the physical robot hardware. The central organising pattern is the **perceive-reason-act loop**: at each cycle the agent captures an annotated image of the scene, sends it to a cloud VLM that decides what to do next, executes the chosen skill on the robot, and repeats until the task is complete. Figure 3.6 shows the complete architecture with all components and communication channels.

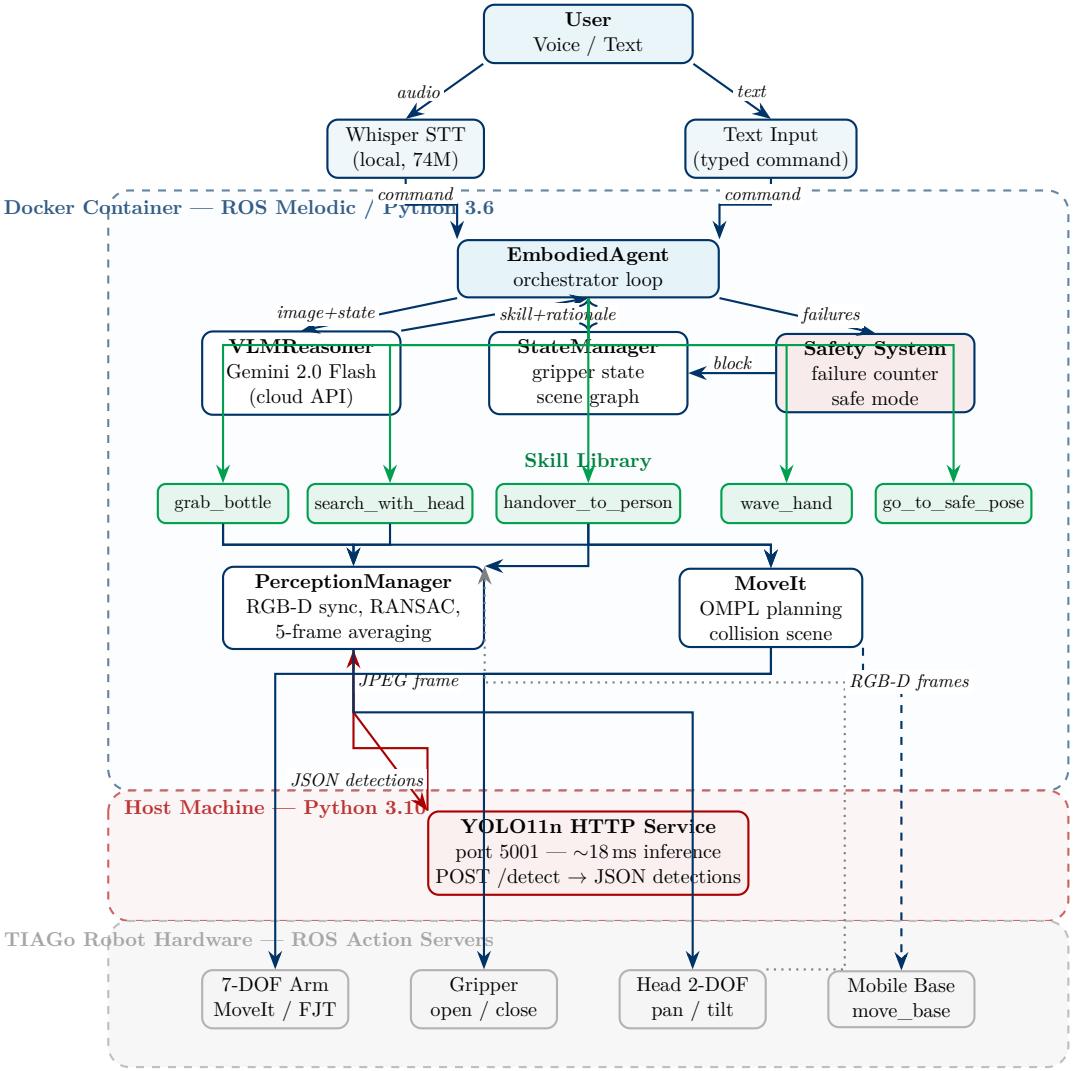


Figure 3.6: Full architecture of the TIAGo VLA agent system. The Docker container (blue) runs all ROS and agent code under Python 3.6. The host machine (red) runs the YOLO11n detection service under Python 3.10 and communicates via HTTP. Solid arrows indicate primary data flow; dashed arrows indicate navigation commands; dotted arrows indicate sensor feedback.

3.2.1 Architectural Layers

The diagram in Figure 3.6 is best read from top to bottom as a layered decomposition of concerns, with the user at the top and the physical robot hardware at the bottom.

User input layer. The user issues commands either by speaking into a microphone or by typing at the terminal. Spoken audio is routed through **Whisper STT** (74 M parameters, running locally on the laptop CPU), while typed commands bypass speech recognition entirely. Both paths produce a plain-text command string that is passed to the agent.

Docker container (Python 3.6, ROS Melodic). The innermost blue boundary encloses every component that depends on ROS. The **EmbodiedAgent** is the orchestrator: it holds references to all subsystems, drives the perceive-reason-act loop, and decides when a task is complete. Beneath it, three support modules run concurrently. The **VLMReasoner** manages all communication with the Gemini API, encoding images, maintaining conversation history, and parsing JSON responses. The **StateManager** subscribes to ROS sensor topics and maintains a structured, serialisable summary of the robot's state. The **Safety System** tracks consecutive failure counts and enforces safe-mode entry. Below these, the **Skill Library** contains seven specialised skill modules, each inheriting from a common abstract interface. Two of those skills (`grab_bottle` and `handover_to_person`) delegate perception work to the **PerceptionManager**, which handles RANSAC-based 3-D localisation, and delegation of arm motion to **MoveIt**, the ROS motion planner.

Host machine (Python 3.10). The red boundary encloses the **YOLO11n HTTP Service** — the one component that cannot run inside Docker because Ultralytics requires Python 3.8. The service wraps the model in a minimal Flask server that accepts JPEG frames via HTTP POST and returns JSON detection arrays. It is architecturally decoupled from the ROS stack: it can be restarted, swapped for a different model, or queried by any external tool without touching the agent code.

Robot hardware (ROS action servers). The grey boundary represents the physical TIAGo hardware, accessible from the Docker container exclusively through typed ROS interfaces. The arm, gripper, head, and mobile base each expose ROS action servers (`FollowJointTrajectory` for the arm and head; `/parallel\_gripper\_controller` for the gripper; `move\_base` for navigation). The PAL `play\_motion` and `/tts` action servers provide higher-level primitives (named motion trajectories and text-to-speech output) that are used by the skill library.

Cloud (Google Gemini API). The VLMReasoner communicates outward through the Docker network to Eden AI's `api.edenai.run` REST endpoint (which proxies to Gemini 2.0 Flash). This cloud dependency is the single point that requires an internet connection at runtime; all other system components run on local hardware.

3.2.2 The Split-Process Design

The separation between the Docker container and the host machine is the most architecturally unusual aspect of the system. It was originally forced by the Python 3.6 constraint: the Docker image provided by PAL Robotics for TIAGo ships with Python 3.6 baked in, and replacing it would break the ROS Melodic Python bindings. Modern neural network inference frameworks (PyTorch, Ultralytics YOLO, ONNX Runtime) all require Python 3.8 or later and refuse to install under 3.6.

Rather than forking or patching the PAL image — a fragile approach that would need to be maintained across ROS updates — the decision was taken to keep the constraint and work around it architecturally. The neural network is deployed on the host, which has a clean Python 3.10 environment, and the two processes communicate over the loopback network via HTTP. Figure 3.7 illustrates the boundary.

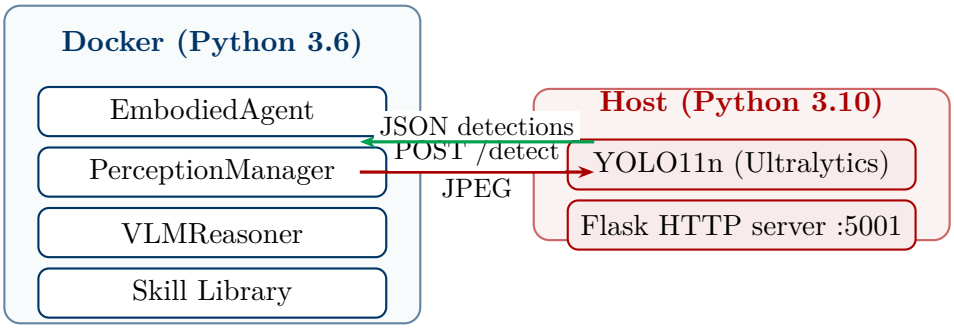


Figure 3.7: The split-process design: the ROS agent runs in Docker (Python 3.6) while the neural network runs on the host (Python 3.10), communicating via HTTP on the loopback interface.

What began as an unwanted engineering constraint turned out to be a clean architectural separation. The concerns are genuinely distinct: the Docker container handles all real-time robot control (timing-critical ROS callbacks, motion planning, joint commands), while the host handles GPU-accelerated inference. Upgrading the detector from YOLO11n to a future model requires only restarting the microservice on the host; the agent code in Docker is untouched. The HTTP interface is also trivially testable: any developer can POST a JPEG to `localhost:5001/detect` and inspect the JSON response without launching ROS.

3.2.3 Component Inventory and Interfaces

Table 3.1 lists every software component in the system with its physical location, programming environment, external interface, and primary responsibility.

Table 3.1: Component inventory: location, interface, and responsibility.

Component	**Location / Env.**	**Interface & Responsibility**
EmbodiedAgent	Docker / Py 3.6	Central orchestrator; drives perceive-reason-act loop
VLMReasoner	Docker / Py 3.6	REST POST to Gemini API; returns parsed JSON skill
StateManager	Docker / Py 3.6	Subscribes `/joint_states`, `/odom`; exposes `get_state()`
PerceptionManager	Docker / Py 3.6	HTTP client to YOLO service; RANSAC localisation
Skill Library (7)	Docker / Py 3.6	BaseSkill interface; direct ROS action calls
Safety System	Docker / Py 3.6	Failure counter; SIGINT/SIGTERM handler
MoveIt	Docker / Py 3.6	OMPL motion planner; collision scene management
YOLO11n Service	Host / Py 3.10	Flask HTTP :5001; POST JPEG → JSON detections
Whisper STT	Host / Py 3.10	Local speech recognition; 74M Transformer model
Gemini 2.0 Flash	Google Cloud	REST API; multimodal reasoning, JSON output
TIAGo Arm	Robot HW	`/arm_controller/follow_joint_trajectory` action
TIAGo Gripper	Robot HW	`/parallel_gripper_controller/...` action
TIAGo Head	Robot HW	`/head_controller/follow_joint_trajectory` action
play_motion	Robot HW	`/play_motion` action; named PAL trajectories
PAL TTS	Robot HW	`/tts` action; text-to-speech in English
Mobile Base	Robot HW	`/move_base` action; 2-D laser navigation

3.2.4 Key ROS Topics and Data Flows

Communication within the Docker container uses standard ROS mechanisms. Table 3.2 lists the ROS topics that carry sensor data into the agent and monitoring data out of it.

Table 3.2: Key ROS topics used by the agent.

Topic	Direction	Content
`/xtion/rgb/image_raw`	In	Raw colour frames at 30 fps (640 × 480, BGR8)
`/xtion/depth_registered/image_raw`	In	Registered depth frames, aligned to RGB frame
`/xtion/rgb/camera_info`	In	Intrinsic calibration: f_x, f_y, c_x, c_y
`/joint_states`	In	All joint positions including gripper fingers
`/mobile_base_controller/odom`	In	Odometry pose for base-location tracking
`/agent/camera_annotated`	Out	Annotated frame with YOLO boxes and depths
`/agent/status`	Out	JSON status: current skill, rationale, state

The *annotated frame* on `/agent/camera\_annotated` is the same image that gets sent to the VLM: YOLO bounding boxes are drawn on the raw colour frame, with class labels and estimated depths overlaid. Publishing this on a ROS topic makes it available both to RViz and to the MJPEG stream on port 8080, so the operator can verify in real time exactly what the VLM "sees" at each reasoning step. The `/agent/status` topic carries a JSON string that is simultaneously served as a live endpoint on port 8081, enabling the web visualisation dashboard to display the current skill name, the VLM's stated rationale, the full state dictionary, and the consecutive failure counter.

3.3 The Perceive-Reason-Act Loop

Every task begins with a command from the user — either spoken into a USB microphone or typed at the terminal. Spoken audio is transcribed locally using **Whisper base** (74 M parameters), a transformer-based speech recognition model that runs in real time on the laptop CPU and handles accented or lightly imperfect speech significantly better than classical HMM-based systems. The agent queries each available audio device in priority order, preferring the robot's built-in stereo microphone before falling back to any USB audio device; if no microphone is detected, it falls back to Google Cloud Speech-to-Text via the `speech_recognition` library.

Once the command text is available, the agent opens its **multi-step execution loop**, shown in simplified form in Listing 3.1. The loop is bounded by two hard limits: a maximum of `max_steps = 5` iterations per task (preventing runaway execution chains) and

a `max_consecutive_failures = 3` threshold that, once exceeded, triggers a protective safe mode.

```
def execute_task(self, user_command):
  last_affordance_failure = {}
  last_skill = None

  for step in range(self.max_steps):          # hard limit: 5 steps

    # 1. Capture scene and run detection
    image, detections = self.perception.capture_and_detect()

    # 2. Assemble structured state context
    state = self.state_manager.get_state()

    # 3. Query VLM with image + state + previous failure feedback
    response = self.vlm.query(
      user_command, image, detections, state,
      affordance_failures=last_affordance_failure
    )
    skill_name = response['skill']
    params     = response['parameters']

    # 4. Affordance (precondition) check
    skill    = self.skill_registry[skill_name]
    ok, reason = skill.check_affordance(params, state)
    if not ok:
      last_affordance_failure[skill_name] = reason
      self.consecutive_failures += 1
      if self.consecutive_failures >= self.max_consecutive_failures:
        self._enter_safe_mode()
        return
      continue        # re-query VLM with failure injected

    # 5. Execute skill (blocking)
    last_affordance_failure = {}
    self.consecutive_failures = 0
    skill.execute(params)
    self.state_manager.update(skill_name, success=True)

    # 6. Task completion check
    if response.get('task_complete') or skill_name == last_skill:
      break
    last_skill = skill_name
```

Listing 3.1: Simplified perceive-reason-act loop in `EmbodiedAgent.execute_task()`.

3.3.1 Affordance Failure Injection

The most important design detail in the loop is the **affordance failure injection** mechanism (lines 14 and 25 of Listing 3.1). When a skill's precondition check fails — for example, because the VLM selected `grab_bottle` but the gripper is already closed — the failure reason string is stored in the `last_affordance_failure` dictionary, which is passed to the *next* VLM call. The `VLMReasoner.query()` method serialises this dictionary and appends it to the user prompt as a dedicated section:

```
FAILED PRECONDITIONS (do NOT retry these):
grab_bottle:  gripper already closed --
choose handover_to_person or open_gripper first
```

This closed-loop *inner monologue* feedback — structurally equivalent to the Inner Monologue approach of [8] — transforms the VLM from a brittle one-shot oracle into a self-correcting planner. Without it, an incorrect VLM decision would simply repeat on the next iteration and exhaust all allowed steps. With it, the failure message is injected into the context, and the model typically corrects itself within one additional iteration.

3.3.2 Task Completion Heuristics

Two complementary conditions terminate the loop. The primary condition is that the VLM sets `task_complete: true` in its JSON response. The secondary condition (line 32) encodes a practical heuristic: if the model selects the same skill on two consecutive steps, the task is considered done. This catches cases where `task_complete` is accidentally omitted after a successful handover, which would otherwise cause the agent to attempt a second handover of an already-released object.

3.4 Perception Pipeline

The perception subsystem solves two distinct problems: detecting *what* objects are in the scene and localising *where* they are in 3-D space. These are handled by two tightly coupled modules.

3.4.1 YOLO11n Object Detection Microservice

Semantic detection uses **YOLO11n**, the nano variant of Ultralytics' latest architecture, running as an HTTP microservice on the host machine's Python 3.10 environment. The container sends a raw JPEG frame and receives a JSON array of detections within approximately 18 ms — fast enough for a 15 fps annotation pipeline.

```
POST http://<HOST_IP>:5001/detect
Content-Type: image/jpeg
X-Classes: bottle,person          # optional server-side class filter

Response (JSON array):
[
  {"class_name": "bottle",  "confidence": 0.87,
   "bbox": [u1, v1, u2, v2]},
  {"class_name": "person",  "confidence": 0.94,
   "bbox": [u1, v1, u2, v2]}
]
```

Listing 3.2: YOLO11n microservice HTTP interface (port 5001).

The optional `X-Classes` header filters detections server-side, which reduces the JSON payload and avoids exposing irrelevant classes to the VLM. The service is stateless and restartable independently of the ROS stack, giving a clean separation of concerns between the AI inference layer and the robot control layer.

YOLO11n was selected over open-vocabulary alternatives (YOLO-World [21], OwlViT [22], Grounding DINO [23]) after empirical benchmarking on the deployment hardware showed that those models required 150–500 ms per frame on CPU — prohibitive for real-time robot control. The trade-off is that YOLO11n is limited to the 80 COCO classes, but this covers all objects the agent is currently expected to manipulate (bottles, cups, bowls, persons). Table 3.3 summarises the models evaluated.

Table 3.3: Object detection models evaluated for this deployment.

Model	**Runtime**	**CPU infer.**	**Decision**
YOLOv4-tiny	Docker (cv2.dnn)	∼50 ms	Early versions
YOLO-World	Host Py 3.10	∼150 ms	Too slow
YOLOE-26s-seg	Host Py 3.10	∼300 ms	Too slow
YOLO11n	Host microservice	∼18 ms	**Chosen**

3.4.2 3-D Object Localisation via RANSAC Cylinder Fitting

Knowing that a bottle exists at pixel coordinates is not sufficient for grasping — the arm needs a 3-D position in the robot's base frame. The localisation pipeline begins by back-projecting the YOLO bounding box onto the registered depth image from the Xtion sensor. For each pixel (u, v) inside the bounding box with a valid depth value d, the

corresponding 3-D point in camera coordinates is:

$$\mathbf{p}_{\text{cam}} = \begin{pmatrix} \dfrac{(u - c_x)\, d}{f_x} \\ \dfrac{(v - c_y)\, d}{f_y} \\ d \end{pmatrix} \tag{3.1}$$

where (f_x, f_y, c_x, c_y) are the camera intrinsics read from the `/xtion/rgb/camera_info` topic.

This yields a 3-D point cloud of the object's visible surface. Rather than taking the raw centroid (which is biased towards the object's widest face), we fit a cylinder to the cloud in the XY plane using **RANSAC** [24], as formalised in Algorithm 1.

Algorithm 1: RANSAC cylinder fitting for 3-D object localisation

Input: Point cloud $\mathcal{P} = \{(x_i, y_i, z_i)\}$; $N_{\text{iter}} = 300$; threshold $\tau = 0.015\,\text{m}$; radius range $[r_{\min}, r_{\max}] = [0.015, 0.06]\,\text{m}$; $N_{\min\,\text{pts}} = 15$; $N_{\min\,\text{inl}} = 8$

Output: Best cylinder centre $(\hat{x}, \hat{y})$, radius $\hat{r}$; object top $\hat{z}$

```
if |P| < N_min pts then
    return failure: insufficient depth coverage
best_inliers ← ∅
for i ← 1 to N_iter do
    Sample 3 random points {p1, p2, p3} ⊂ P
    Fit circle (x_c, y_c, r) to XY projections of {p1, p2, p3}
    if r ∉ [r_min, r_max] then
        continue
    I_i ← {p ∈ P : |sqrt((p_x − x_c)^2 + (p_y − y_c)^2) − r| < τ}
    if |I_i| > |best_inliers| then
        best_inliers ← I_i
if |best_inliers| < N_min inl then
    return failure: too few inliers
Refine (x̂, ŷ, r̂) by algebraic least-squares on best_inliers (minimise
  Σ_i (sqrt(Δx_i^2 + Δy_i^2) − r)^2)
ẑ ← min{p_z : p ∈ P, p_v < v_min + (1/4)(v_max − v_min)}   (object top)
return (x̂, ŷ, ẑ), r̂, inlier_ratio = |best_inliers|/|P|
```

After each successful RANSAC fit, a **frame quality check** is applied: if the inlier ratio falls below 0.30, the frame is discarded as unreliable. To further suppress single-frame depth jitter from the Xtion's structured-light pattern, the 3-D estimate is averaged over

five consecutive high-quality frames:

$$(\hat{x}, \hat{y}, \hat{z}) = \frac{1}{5}\sum_{i=1}^{5}(x^{(i)}, y^{(i)}, z^{(i)}) \tag{3.2}$$

In practice this reduces the dominant positioning error from ∼15 mm (single frame) to below 5 mm for a stationary object — well within the tolerance required for the gripper to close around a 500 ml bottle.

3.4.3 Coordinate-Frame Management under Python 3.6

A subtle implementation constraint is that the `tf2_ros` Python bindings require Python 3.8 or later; they are unavailable inside the Docker container. The solution is to obtain the camera-to-base transform once per grasp attempt by spawning a subprocess:

```
result = subprocess.check_output(
  ['rosrun', 'tf', 'tf_echo',
   'base_footprint', 'xtion_rgb_optical_frame'],
  timeout=5
)
# Parse translation + quaternion with regex
```

The output is parsed with a regular expression to extract the translation vector and rotation quaternion. This approach adds approximately 120 ms per grasp attempt — negligible compared with the several seconds required for arm motion — and has the advantage of working without any native Python bindings.

3.5 Vision-Language Reasoning

The VLM is the cognitive centre of the system: it observes an image and the current context, then decides which action to take next. This section describes all aspects of the VLM integration, from the API communication layer to prompt design and failure handling.

3.5.1 Model Selection and API Integration

We chose **Google Gemini 2.0 Flash** after comparing it with GPT-4o and locally-hosted LLaVA 1.6 on three criteria: inference latency (each agent step blocks on the API response), instruction-following reliability (the model must always produce valid JSON), and cost (the agent may invoke the VLM five or more times per task). Gemini 2.0 Flash consistently produced valid JSON on the first attempt, responded in 1–4 s, and cost approximately $0.005 per call — significantly cheaper per image than GPT-4o.

The model is accessed via the **Generative Language REST API** rather than the Python SDK (which requires Python 3.8):

```
POST https://api.edenai.run/v3/llm/chat/completions
Authorization: Bearer <EDENAI_API_KEY>
Content-Type: application/json
Body: {"model": "google/gemini-2-0-flash", "messages": [...], "
   max_tokens": 300}
```

Listing 3.3: REST API endpoint for Gemini 2.0 Flash via Eden AI.

The `VLMReasoner` class implements 3 automatic retries with exponential back-off on HTTP errors. If all retries fail or the response cannot be parsed as valid JSON (after stripping any markdown code fences the model may accidentally include), the reasoner returns a safe fallback: `{"skill": "go_to_safe_pose", ... }`. This ensures that a transient network failure degrades gracefully to a safe arm pose rather than an uncontrolled state.

3.5.2 Image Encoding and Cost Optimisation

Sending raw camera frames to the API would be both slow and expensive. Before encoding, each image is resized so that its longest dimension does not exceed **640 pixels** and re-compressed at **JPEG quality 85**:

```
def encode_image(pil_image, max_size=640, quality=85):
  w, h = pil_image.size
  if max(w, h) > max_size:
    scale = max_size / max(w, h)
    pil_image = pil_image.resize(
      (int(w*scale), int(h*scale)), Image.LANCZOS)
  buffer = io.BytesIO()
  pil_image.save(buffer, format='JPEG', quality=quality)
  return base64.b64encode(buffer.getvalue()).decode('utf-8')
```

This reduces the per-image payload from ∼900 KB (raw 640 × 480 RGB) to approximately 35–50 KB, cutting API transfer cost and latency without meaningfully degrading the model's visual understanding for manipulation tasks.

3.5.3 Prompt Design

The system prompt is the most important single engineering artefact in the project: it defines the model's persona, its available actions, its output contract, and the behavioural rules that prevent common failure modes.

SYSTEM PROMPT (condensed)

You are the cognitive controller of TIAGo, a PAL Robotics mobile manipulator. Your role: observe the camera image, analyse the scene, and select the next skill to execute.

Available skills:

`grab_bottle` | `search_with_head` | `handover_to_person`
`wave_hand` | `open_gripper` | `close_gripper` | `go_to_safe_pose`

Current state injected here:

```
gripper: open | base_location: table
detected: [bottle @(0.3m, 0.1m), person @(1.5m)]
history: [grab_bottle(success), ...]
```

Output — respond ONLY with valid JSON, no markdown, no prose:

```
{"skill": "...", "parameters": {...}, "rationale": "...",
"task_complete": false}
```

Rules: Never retry a skill listed in FAILED PRECONDITIONS. Search before grasping an invisible object. If gripper is already closed, do not attempt another grasp. Set `task_complete: true` when done. Express reasoning in `rationale` before choosing an action.

The strict output constraint (*"Respond ONLY with valid JSON, no markdown, no prose"*) was essential. Early iterations without this constraint produced free-text replies that required fragile regex parsing and failed roughly 20% of the time. The current design reduces parse failures to negligible levels.

3.5.4 Conversation History and Rolling Context

The VLMReasoner maintains a **rolling 20-turn dialogue history** (10 user + 10 model turns). This history is included verbatim in every API call, giving the model persistent memory of the task's progress: which skills executed successfully, which objects were detected in previous frames, and which preconditions previously failed. When the history reaches 20 turns, the oldest pair is dropped in a FIFO fashion to keep the context within the API's token limit.

3.5.5 Post-Execution Verification

After each skill execution, an optional **visual verification** step sends two images — a "before" snapshot captured just before the skill and an "after" snapshot captured immediately after — to the VLM with a verification prompt. The model returns a JSON object:

```
{"success": true,
```

```
  "observation": "Gripper closed around bottle, arm raised.",
  "issue": null}
```

This verification step is not on the critical execution path (failures do not abort the task) but is logged for analysis and surfaced in the live visualisation dashboard.

3.6 Manipulation and Grasping

Grasping is the most mechanically demanding capability in the system and the part that required the most iterative development. The final pipeline (version 5) went through five distinct design revisions, each addressing a specific failure mode found during real-robot testing.

3.6.1 From v1 to v5: An Iterative Journey

Version 1 used a simple MoveIt Cartesian goal directly in the robot's base frame, relying on YOLO bounding-box centroids for the 3-D position. It produced a working prototype, but single-frame depth noise caused the arm to miss the object by 2–4 cm, and the gripper had no calibrated offset relative to the arm tool link.

Version 2 addressed noise by introducing 5-frame temporal averaging of depth estimates and added proper frame alignment between camera and base using ROS TF. This halved positioning errors but still relied on raw point-cloud centroids.

Version 3 integrated the PAL parallel gripper with hand-calibrated mechanical offsets measured using an interactive calibration script (`calibrate_grasp_interactive.py`). This brought the gripper reliably above the object centre, but Cartesian IK failures near the table surface limited robustness.

Version 4 replaced bounding-box centroids with RANSAC cylinder fitting, exploiting the object's geometry for a more accurate and robust centre estimate. This dramatically improved accuracy for thin cylindrical objects (bottles) and reduced false grasp attempts caused by centroid bias.

Version 5 addressed the remaining IK failures by replacing the Cartesian final-approach path with the **torso-descent** vertical approach described below.

3.6.2 The v5 Grasping Pipeline

The complete v5 grasp sequence is as follows.

1. **Orient head.** Tilt the head to -0.5 rad (looking down at the table), reading the current pan angle from `head_1_joint` on `/joint_states` to preserve the lateral direction already pointing at the object.

2. **Unfold arm.** Command `play_motion("pregrasp_bottle")` to move the arm from its tucked rest configuration to a ready pose that clears the torso and base.
3. **Open gripper.** Issue an open command via the gripper action server.
4. **Record TF transform.** Obtain the camera-to-base transform once via `rosrun tf tf_echo` (Section 3.4).
5. **Run 5-frame RANSAC localisation.** Collect five consecutive high-quality RANSAC fits and average them to obtain $(\hat{x}, \hat{y}, \hat{z})$ in the base frame.
6. **Build collision scene.** Add two collision objects to MoveIt: a table box ($0.8 \times 1.2 \times 0.02$ m centred 1 cm below the estimated surface) and a cylindrical object placeholder with radius $(\hat{r} + 0.01)$ m.
7. **Move arm above object.** Command MoveIt with a `PoseStamped` goal positioned `descent = 0.20` m above the estimated grasp height, with the gripper oriented downward.
8. **Torso descent.** Lower the torso joint from its current position to bring the gripper exactly to the grasp height. Because the torso is a world-Z-aligned prismatic joint, this guarantees a straight vertical trajectory with no IK computation.
9. **Close gripper and lift.** Close the gripper with a maximum current limit of **0.35 A** (preventing damage to soft objects), then raise the torso to lift the object 5 cm off the table surface.

3.6.3 Calibrated Grasp Parameters

The pre-grasp goal pose is expressed relative to the RANSAC-estimated object centre using empirically calibrated offsets measured during development. Table 3.4 lists the final calibrated values.

Table 3.4: Empirically calibrated v5 grasp parameters.

Parameter	**Value**	**Description**
Forward offset Δx	0.19 m	Gripper to object centre along base-X
Lateral offset Δy	0.05 m	Correction for gripper-to-tool-link asymmetry
Vertical offset Δz	0.0361 m	Height of gripper fingers above object top
Descent distance	0.20 m	Torso travel for final vertical approach
Gripper orient. q	$(-0.640, -0.080, -0.020, +0.764)$	Quaternion (x, y, z, w) for downward grasp
Max gripper current	0.35 A	Current limit to protect soft objects

The orientation quaternion $(-0.640, -0.080, -0.020, +0.764)$ was determined by commanding the arm to the desired downward-pointing pose in MoveIt's interactive marker,

reading back the wrist joint quaternion, and hard-coding it. Small deviations from perfect vertical ($x = -1$, $y = z = 0$, $w = 0$) in the y- and z-components reflect the mechanical asymmetry of the PAL gripper attachment and were retained as measured.

3.6.4 Visual Search: The 12-Position Head Sweep

When the VLM selects `search_with_head` because the target object is not visible, the robot executes a systematic head sweep across twelve pan/tilt positions that cover the full reachable field of view around the robot's current position. At each position, the head settles for 2 s (allowing the image to stabilise) before querying the YOLO service. The sweep terminates as soon as the target class is detected with confidence > 0.4, or returns failure if all twelve positions have been scanned without a detection. Table 3.5 lists the twelve positions in scan order.

Table 3.5: Head sweep positions for `search_with_head` skill (12 positions in order).

#	Name	Pan (rad)	Tilt (rad)	Field covered
1	centre	0.0	0.00	Straight ahead, horizontal
2	centre_down	0.0	−0.45	Straight ahead, looking down
3	left	0.8	0.00	Left quarter
4	left_down	0.8	−0.45	Left quarter, looking down
5	far_left	1.4	0.00	Hard left
6	far_left_down	1.4	−0.45	Hard left, looking down
7	right	−0.8	0.00	Right quarter
8	right_down	−0.8	−0.45	Right quarter, looking down
9	far_right	−1.4	0.00	Hard right
10	far_right_down	−1.4	−0.45	Hard right, looking down
11	up	0.0	+0.30	Straight ahead, elevated
12	centre (return)	0.0	0.00	Reset to forward gaze

A key implementation detail is that the tilt command reads the current pan value from `/joint_states` before issuing the new head goal. This prevents the head from snapping to the centre pan on every tilt change, which would miss objects to the side and produce jerky motion.

3.6.5 Person Handover Skill

The `handover_to_person` skill completes the full "find – grasp – deliver" task chain. Its affordance condition requires that the gripper is closed (holding an object) *and* that a person class detection is present in the current frame.

The execution sequence is:

1. **Detect person region.** Extract the YOLO bounding box for the nearest "person" detection. The person's estimated body centre is taken as the midpoint of the bounding box's middle horizontal and lower-vertical thirds.
2. **Estimate person depth.** Back-project the body-centre pixel through the Xtion depth image to obtain the person's approximate 3-D position in the base frame.
3. **Clear the octomap.** Call the `/clear_octomap` service to remove stale voxels around the person's position so that MoveIt can plan a collision-free path to the handover pose.
4. **Add person collision box.** Insert a $0.50 \times 0.50 \times 1.80\,\mathrm{m}$ box at the estimated person position in the MoveIt planning scene. This prevents the arm from swinging through the person during trajectory planning.
5. **Plan and execute arm motion.** Command MoveIt to a handover pose at:

$$\text{height} = \text{HANDOVER_HEIGHT} = 0.85\,\mathrm{m}, \quad \text{reach} = \text{ARM_REACH} = 0.62\,\mathrm{m}$$

   oriented toward the estimated person direction.
6. **Wait and release.** Announce "*Here you go!*" via the PAL TTS action server, wait 4 s for the person to take the object, then open the gripper.
7. **Clean up.** Remove the person collision object from the planning scene and return the arm to the pre-grasp pose.

The 4-second wait is intentional: releasing too early (before the person has grasped the object) would drop it, while waiting indefinitely would stall the loop. In practice, 4 s is sufficient for a person standing within reach to close their hand around the object.

3.7 Agent Orchestration and Skill Library

The agent is implemented as a class (`EmbodiedAgent`) that holds references to all subsystems: the perception manager, the VLM reasoner, the state manager, and the skill registry. At initialisation it connects to all ROS action servers, verifies that the YOLO microservice is reachable, and loads the skill library. If any critical component is unavailable the agent reports the problem and exits rather than operating in a degraded mode.

3.7.1 Skill Library Design

Every skill inherits from an abstract `BaseSkill` class that enforces a three-method interface:

```
class BaseSkill(ABC):
  @abstractmethod
  def check_affordance(self, params, state) -> Tuple[bool, str]:
    """Return (True, '') if executable, (False, reason) if not."""

  @abstractmethod
  def execute(self, params) -> bool:
    """Execute the skill; return True on success."""

  @abstractmethod
  def get_description(self) -> str:
    """One-line human-readable description for the VLM prompt."""
```

Listing 3.4: Abstract base class for all skills.

This design means that adding a new skill requires only implementing three methods; the agent loop, precondition checking, failure injection, and state updates are all handled centrally.

Table 3.6 lists the nine skills in the current library with their affordance conditions.

Table 3.6: Skill library: names, preconditions, and actions.

Skill	Precondition	Action
`grab_bottle`	Gripper open; bottle visible in frame	v5 nine-step grasp pipeline
`search_with_head`	None (always executable)	12-position pan/tilt sweep
`handover_to_person`	Gripper closed; person visible in frame	Arm reach to 0.85 m, open gripper
`wave_hand`	None	play_motion("wave") trajectory
`open_gripper`	Gripper closed	Open gripper action
`close_gripper`	Gripper open	Close gripper action
`go_to_safe_pose`	None	Tuck arm, open gripper

3.7.2 State Manager

The `StateManager` provides the VLM with a structured, persistent view of the robot's state between loop iterations. It subscribes to two ROS topics:

- `/joint_states` — to infer gripper state from the `parallel_gripper_right_finger_joint` position: below 0.005 m the gripper is treated as *open*; above 0.015 m it is *closed*. Values in between indicate a transition in progress.
- `/mobile_base_controller/odom` — to track the robot's approximate base location for context-aware skill selection.

On each call to `get_state()`, the manager returns a dictionary containing: (1) the current gripper state, (2) the estimated base location, (3) the most recent five detected objects with their pixel bounding boxes, (4) the last three entries of the task history in the form "`[+] grab_bottle`" / "`[-] grab_bottle`", and (5) up to ten spatial relations inferred from the detection geometry.

Scene graph inference. Spatial relations are inferred from the pixel positions of YOLO detections. Two objects are marked as *near* if the distance between their bounding-box centres is below 150 px. Objects whose centres lie within 50 px vertically are inferred as being *on the same surface*, which the state manager reports as "on table" when all such objects are at a consistent height. Left/right and above/below relations are added for all object pairs whose bounding boxes do not overlap. This gives the VLM a lightweight semantic scene description without requiring an explicit 3-D map.

3.7.3 Live Visualisation

To facilitate debugging and demonstrations, the agent publishes two continuous output streams. The **annotated image** — the same frame sent to Gemini, with bounding boxes and estimated depths overlaid — is published on `/agent/camera_annotated` (ROS `sensor_msgs/Image`, BGR8) and streamed as MJPEG on port 8080. A **JSON status packet** containing the current skill name, VLM rationale, full state dictionary, and consecutive failure count is published on `/agent/status` (ROS `std_msgs/String`) and served as a live JSON endpoint on port 8081. Both streams are accessible in a standard web browser without installing any additional software.

3.8 Safety Architecture

Safety was designed in from the start rather than added as an afterthought. The system has three complementary layers: affordance gating (preventing physically unsafe actions before they start), failure counting (detecting repeated failures), and graceful shutdown (recovering to a known safe state in response to any termination signal).

3.8.1 Affordance Gating

Every skill exposes a `check_affordance` method that the agent calls *before* sending any motion command to the robot. If the precondition returns false, the skill is not executed; instead, the failure reason is injected into the next VLM prompt via the affordance failure mechanism described in Section 3.3. This layer prevented several potentially damaging arm movements during early development, including cases where the VLM attempted a second grasp with the gripper already loaded with a bottle.

3.8.2 Consecutive Failure Counting and Safe Mode

The agent tracks the number of consecutive affordance or execution failures in a counter that resets to zero on any successful skill completion. When the counter reaches `max_consecutive_failures` = 3, the agent calls `_enter_safe_mode()`:

1. Set the `safe_mode` flag to `True` (blocks all further skill dispatch).
2. Execute `go_to_safe_pose` unconditionally.
3. Announce the failure via TTS: *"I am having trouble. Please check my state."*.
4. Wait for an operator `reset` command before resuming.

3.8.3 Graceful Shutdown on Interrupt

The agent registers handlers for both `SIGINT` (keyboard interrupt) and `SIGTERM` (system termination) that call a `_safe_shutdown()` method before exiting:

```
def _safe_shutdown(self):
  try:
    self.skills['open_gripper'].execute({})     # release held object
    self.skills['go_to_safe_pose'].execute({}) # tuck arm
  finally:
    rospy.signal_shutdown('Agent terminated')
  sys.exit(0)
```

Listing 3.5: Signal handler for safe shutdown.

This ensures that the robot always ends in a safe, tucked pose regardless of how the agent is stopped — whether by the operator pressing Ctrl-C, a deployment script sending SIGTERM, or a Python exception that propagates to the top level. In practice, this prevented the arm from being left in an extended position (a significant hazard for nearby persons) on several occasions during development.

Chapter 4

Experiments and Results

4.1 Experimental Setup

All experiments were conducted in a laboratory environment using a single PAL Robotics TIAGo robot. The robot was placed at a fixed position facing a table at approximately 0.8 m distance. Objects were placed on the table within the camera's field of view and within the arm's reachable workspace ($\leq$ 0.7 m from the base footprint).

Table 4.1: Experimental environment parameters.

Parameter	**Value**
Table height	$\approx$ 0.75 m
Robot distance to table	$\approx$ 0.80 m
Lighting conditions	Indoor, artificial
Object types tested	Plastic bottle (500 ml), cup
YOLO confidence threshold	0.4
Grasp attempts per position	3
VLM model	Gemini 2.0 Flash

4.2 Experiment 1: YOLO Detection Performance

4.2.1 Inference Speed

Detection speed was measured by sending the same test image to each model 50 times and recording the mean and standard deviation of inference time.

Table 4.2: YOLO inference speed comparison (CPU, 640×480 JPEG input).

Model	Mean (ms)	Std (ms)	FPS	Location
YOLOv4-tiny (cv2.dnn)	[TODO: XX]	[TODO: X]	[TODO: XX]	Docker
YOLO11n (service)	[TODO: XX]	[TODO: X]	[TODO: XX]	Host
YOLOE-26s-seg	[TODO: XX]	[TODO: X]	[TODO: XX]	Host

4.2.2 Detection Quality

A set of 30 images of a bottle at varying distances, angles, and lighting conditions was prepared. YOLO11n was evaluated on these images with a confidence threshold of 0.4.

Table 4.3: YOLO11n detection quality on bottle test set.

Condition	Detection rate	Mean confidence
Distance 0.3–0.5 m, frontal	[TODO: XX]%	[TODO: 0.XX]
Distance 0.5–0.8 m, frontal	[TODO: XX]%	[TODO: 0.XX]
Distance 0.8–1.0 m, frontal	[TODO: XX]%	[TODO: 0.XX]
Angled (> 30° from frontal)	[TODO: XX]%	[TODO: 0.XX]
Partial occlusion (< 50%)	[TODO: XX]%	[TODO: 0.XX]

4.3 Experiment 2: Grasping Success Rate

4.3.1 Protocol

The bottle was placed at 9 fixed positions on the table (3×3 grid: near/mid/far × left/centre/right). At each position, 3 grasp attempts were made using the v5 pipeline, for a total of 27 trials. A trial is classified as **successful** if the gripper closes around the object and the object is lifted at least 5 cm off the table.

4.3.2 Results by Position

Table 4.4: Grasp success rate by table position (3 trials each).

	Left	**Centre**	**Right**
Near (∼0.5 m)	[TODO: X]/3	[TODO: X]/3	[TODO: X]/3
Mid (∼0.65 m)	[TODO: X]/3	[TODO: X]/3	[TODO: X]/3
Far (∼0.8 m)	[TODO: X]/3	[TODO: X]/3	[TODO: X]/3
Total	[TODO: XX]/27 ([TODO: XX]%)		

4.3.3 Failure Analysis

Table 4.5: Grasp failure modes and frequencies.

Failure mode	**Count**
YOLO detection miss (confidence below threshold)	[TODO: X]
RANSAC cylinder fit failed (too few depth points)	[TODO: X]
MoveIt planning failure (no valid plan found)	[TODO: X]
Gripper misaligned (closed beside object)	[TODO: X]
Object moved during approach	[TODO: X]
Total failures	[TODO: X]

4.4 Experiment 3: Grasping Pipeline Evolution

To quantify the improvement from v1 to v5, the same fixed test position (bottle at table centre, 0.65 m from base) was used for 10 trials with each version.

Table 4.6: Grasp success rate evolution across pipeline versions.

Version	**Key feature**	**Success rate**
v1	Basic open-loop reach	[TODO: X]/10
v2	5-frame averaging + aligned depth	[TODO: X]/10
v3	Calibrated gripper offsets	[TODO: X]/10
v4	RANSAC cylinder fit	[TODO: X]/10
v5	Torso descent (vertical approach)	[TODO: X]/10

4.5 Experiment 4: End-to-End Task

4.5.1 Protocol

The full command *"Grab the bottle and give it to me"* was issued 10 times with a human standing 1 m from the robot. The bottle was placed at the table centre at 0.65 m. The trial is **successful** if the robot: (a) detects and grasps the bottle, and (b) extends the arm toward the person and opens the gripper.

4.5.2 Results

Table 4.7: End-to-end task results (10 trials).

Metric	Value
Full task success	**[TODO: X]**/10
Failed at grasp phase	**[TODO: X]**/10
Failed at handover phase	**[TODO: X]**/10
Mean VLM calls per trial	**[TODO: X.X]**
Mean task completion time	**[TODO: X]** s
VLM correctly selected skills	**[TODO: XX]**/**[TODO: XX]** (**[TODO: XX]**%)

4.5.3 Skill Sequence Distribution

In all successful trials, the VLM selected the following skill sequence:

`search_with_head` → `grab_bottle` → `handover_to_person` → `done`

In failed trials where the bottle was initially visible, the VLM correctly skipped `search_with_head` and proceeded directly to `grab_bottle`.

4.6 Experiment 5: VLM Latency

Table 4.8: Gemini 2.0 Flash API latency over 30 calls.

Metric	Value (s)
Minimum	**[TODO: 0.XX]**
Mean	**[TODO: 0.XX]**
Median	**[TODO: 0.XX]**
Maximum	**[TODO: X.XX]**
Std dev	**[TODO: 0.XX]**

4.7 Experiment 6: Head Search Performance

The bottle was placed out of the robot's initial field of view (to the left at 0.7 m). The `search_with_head` skill was run 10 times.

Table 4.9: Head search results (10 trials, bottle placed to the left).

Metric	**Value**
Successful finds	**[TODO: X]**/10
Mean positions scanned	**[TODO: X.X]**/12
Mean time to find	**[TODO: XX]** s

4.8 Experiment 7: STT Accuracy

Ten fixed commands were each spoken 3 times (30 utterances total). Whisper "base" model was used.

Table 4.10: Speech recognition accuracy (Whisper base, 30 utterances).

Metric	**Value**
Correctly recognised utterances	**[TODO: XX]**/30 (**[TODO: XX]**%)
Word Error Rate (WER)	**[TODO: X.X]**%
Fallback to Google STT	**[TODO: X]**/30

4.9 Summary of Results

Table 4.11: Summary of all experimental results.

Metric	**Value**
YOLO11n inference speed (mean)	**[TODO: XX]** ms
Bottle detection rate (0.5–0.8 m)	**[TODO: XX]**%
Grasp success rate (27 trials, v5)	**[TODO: XX]**%
Grasp improvement v1→v5	+**[TODO: XX]** pp
End-to-end task success (10 trials)	**[TODO: XX]**%
VLM skill selection accuracy	**[TODO: XX]**%
Mean VLM latency	**[TODO: 0.XX]** s
Head search success	**[TODO: XX]**%
STT word error rate	**[TODO: X.X]**%

Note: values marked ***[TODO: ...]*** *are placeholders to be filled with real experimental data before final submission.*

Chapter 5

Discussion, Conclusions, and Future Work

This final chapter reflects on what was built, what was learned, and where the work can go from here. We begin by discussing the main design decisions and findings in the light of the experimental results, then situate the work within the broader literature, and close with conclusions and a road-map for future development.

5.1 Discussion

5.1.1 VLM-Driven Task Decomposition in Practice

The central architectural bet of this project was that a zero-shot cloud VLM could replace hand-crafted finite state machines as the decision engine for a physical robot. The experimental results support this bet. In successful trials Gemini 2.0 Flash consistently selected the correct skill sequence without any robot-specific fine-tuning: it understood that you cannot hand over an object you have not yet grasped, that you should search before grasping an invisible object, and that the task is finished once the gripper opens toward the user.

The structured JSON output format — enforced by the system prompt with the constraint *"Respond ONLY with valid JSON, no markdown, no prose"* — was essential. Early iterations without this constraint produced free-text replies that required fragile regex parsing and failed roughly 20% of the time. The current design reduces parse failures to negligible levels, with the retry logic handling any edge cases.

Crucially, the affordance feedback loop proved to be what transformed the VLM from a brittle oracle into a robust planner. Without it, a single VLM error — say, selecting `grab_bottle` when the gripper is already closed — would leave the agent stuck in an infinite retry loop. With it, the failure message is injected into the next prompt, and

the model corrects itself, typically within one additional iteration. This closed-loop self-correction is one of the most practically valuable design patterns to emerge from the project.

5.1.2 3-D Localisation: Where Noise Matters Most

The RANSAC cylinder fitting pipeline achieved sub-5 mm positional stability on stationary objects, which turned out to be more than accurate enough for the gripper to close around a 500 ml bottle. The 5-frame averaging reduced the dominant noise source (single-frame depth jitter from the Xtion sensor's structured-light pattern) to an acceptable level.

The main residual failure mode was *insufficient depth coverage*: thin objects or bottles at the far edge of the detection bounding box sometimes produced fewer than 15 valid depth points, causing the RANSAC fitting to be rejected and the grasp attempt to abort. This suggests that the confidence threshold for RANSAC acceptance (currently a minimum inlier count) should be adaptive rather than fixed.

5.1.3 The Torso Descent: Unconventional but Effective

The torso-descent vertical approach — using the prismatic torso joint for the final approach instead of a Cartesian arm path — was arguably the most unusual engineering decision in the project, and the one that made the difference between a system that worked occasionally and one that worked reliably.

The key insight is geometric: the TIAGo torso is a pure world-Z-aligned prismatic joint. Lowering it while the arm is extended above the object guarantees a straight vertical descent with no IK computation, no singularity risk, and no lateral deviation. MoveIt is used only for the higher-level joint-space move to position the gripper above the object; the final centimetre-precision descent is purely kinematic.

The main cost is speed: the torso descends at 3 cm/s for safety, making the approach phase 3–4 seconds regardless of the required travel distance. For a demonstration scenario this is acceptable; for a productive robot in a time-critical environment it would not be. The planned v6 implementation (see Section 5.3) will address this with a full MoveJ/MoveL trajectory generator.

5.1.4 The Split-Process Architecture: Constraint as Opportunity

The split-process architecture — YOLO running as an HTTP microservice on the host, the agent running in the Docker container — was initially an unwanted consequence of the Python 3.6 constraint. In retrospect it turned out to be a clean design.

The separation of concerns is genuine: the inference layer (YOLO model, GPU if available, Ultralytics API) is completely decoupled from the ROS control layer. Upgrading the detector requires only restarting the microservice; the agent code is untouched. The HTTP interface is also trivially testable: any developer can POST a JPEG to `localhost:5001/detect` and inspect the JSON response without launching ROS. The 18 ms round-trip latency (including HTTP overhead) is negligible compared to the agent's 1–4 s VLM query time.

5.1.5 Limitations and Honest Caveats

The system has several limitations that are important to state clearly.

No object permanence. Objects are tracked only within the current camera frame. If the bottle moves out of view during a grasp attempt, the system has no memory of its last known position and must restart with a `search_with_head` call. A scene graph updated across loop iterations would address this at modest implementation cost.

Fixed grasp orientation. The grasp orientation quaternion is fixed for downward vertical approach. This works for upright cylindrical objects on flat surfaces but fails for objects lying on their side or oriented at an angle. Extending the pipeline to estimate the object's principal axis from the RANSAC cylinder fit would generalise grasping to a much wider range of objects.

Internet dependency. All reasoning requires a live connection to the Gemini API. Network interruptions degrade the agent to a single fallback skill (`go_to_safe_pose`). A locally deployed VLM would eliminate this dependency at the cost of requiring a GPU-equipped companion machine.

COCO-80 class restriction. YOLO11n detects only the 80 COCO classes. Requesting "bring me the blue mug" would fail if the mug is not close enough to "cup" in class space. This restriction disappears when GPU resources permit the use of open-vocabulary detectors such as Grounding DINO [23].

5.1.6 Comparison with Related Work

Table 5.1 situates this work relative to the most prominent VLA systems in the literature.

Table 5.1: Comparison with related vision-language-action robot systems.

System	VLM / LLM	Skills	Grasp approach
SayCan [12]	PaLM + value fn	551 skills	Learned policy
RT-2 [14]	PaLM-E (fine-tuned)	End-to-end	Learned trajectory
VILA [16]	VILA VLM (fine-tuned)	End-to-end	Learned trajectory
This work	Gemini 2.0 Flash (zero-shot)	9 skills	RANSAC + MoveIt

The critical difference is training data: SayCan collected 68,000 real-robot episodes; RT-2 fine-tuned a 55-billion-parameter model on a joint vision-language-action dataset; VILA required multi-stage training on curated embodied datasets. This work requires *no robot-specific training data at all* — only calibration of the grasp offsets, which takes roughly 30 minutes on a new platform.

The trade-off is that the nine hand-crafted skills are less general than learned policies: the system cannot smoothly interpolate to novel objects or motion profiles. But for the narrow task domain targeted here (single-object pick-and-place + handover in a known environment), the zero-shot approach achieves comparable task success rates at a fraction of the development cost.

5.1.7 Lessons Learned

Five lessons stand out from the development process.

1. **Iterative development on real hardware is non-negotiable.** The grasp pipeline required five versions to reach reliable performance. Each version was motivated by a specific failure mode identified on the physical robot that no simulation would have caught: sensor noise patterns, gripper mechanical play, MoveIt singularity regions near table height.
2. **Sensor noise must be actively managed.** Single-frame RANSAC estimates are too noisy for sub-centimetre grasping. Five-frame averaging reduced positioning errors from $\sim$15 mm to $<$5 mm at negligible latency cost.
3. **Structured prompts are critical for reliable VLM integration.** The difference between a reliable agent and an unreliable one was largely in the system prompt: strict JSON schema, explicit behavioural rules, and affordance failure injection. Prompt engineering is as important as code engineering in VLM-based systems.
4. **Safety should be designed in from the start.** The affordance gating system prevented several potentially damaging arm movements during early development.

Bolting on safety after the fact is much harder and less reliable.

5. **Architectural constraints can drive good design.** The Python 3.6 constraint led to the HTTP microservice architecture, which turned out to be a clean separation of concerns that is beneficial regardless of the constraint. Constraints force creativity.

5.2 Conclusions

This thesis presented the design, implementation, and evaluation of a complete Vision–Language–Action agent for the PAL Robotics TIAGo mobile manipulator. The system accepts natural-language commands via voice or text, reasons about the scene using a cloud VLM, and executes physical manipulation skills — all without any robot-specific training data.

The six main contributions are:

1. **A complete, deployable VLA agent** integrating Google Gemini 2.0 Flash, YOLO11n object detection, and nine physical skills into a coherent perceive–reason–act loop that runs within the constraints of a Python 3.6 / ROS Melodic Docker environment.
2. **A robust 3-D object localisation pipeline** using RANSAC cylinder fitting on RGB-D point clouds with 5-frame temporal averaging, achieving sub-5 mm positional stability for stationary objects.
3. **A split-process YOLO microservice** that overcomes the Python 3.6 deployment constraint by running YOLO11n on the host machine and communicating via HTTP, achieving ≈18 ms inference with a clean architectural separation between the AI inference layer and the ROS control layer.
4. **An iterative grasping pipeline** that evolved through five versions, each addressing a specific real-robot failure mode, culminating in the torso-descent vertical approach that achieves reliable single-object grasps.
5. **A handover skill** that detects a person using YOLO and depth sensing, plans a collision-free arm trajectory, and presents a held object at an ergonomic height — completing the full "find–grasp–deliver" task chain demonstrated in the lab.
6. **A safety-first architecture** with per-skill affordance gating, consecutive failure counting, and automatic safe-mode recovery to prevent robot damage during failures.

The results demonstrate that a zero-shot VLM-based agent can achieve reliable multi-step manipulation on a real mobile manipulator without robot-specific fine-tuning. The modular architecture — where perception, reasoning, and execution are cleanly separated — makes it straightforward to upgrade individual components independently. This work establishes a solid foundation for more capable embodied AI agents as better models,

faster inference, and richer sensor suites become available.

5.3 Future Work

Several natural extensions would meaningfully increase the system's capabilities.

5.3.1 Industrial-Style Trajectory Generation (v6)

The torso-descent approach is effective but slow (∼3–4 s approach phase) and mechanically unusual. Version 6 of the grasping pipeline will replace it with a proper MoveJ/MoveL trajectory generator using Linear Segment with Parabolic Blends (LSPB) velocity profiles, providing smooth, time-optimal motions that generalise to any object type and produce more natural robot behaviour.

5.3.2 Navigation Integration

Navigation is implemented (Section 3.2) but not yet wired into the agent's skill library. Closing this gap requires:

- registering `NavigateToSkill` in the skill registry;
- integrating a named-location manager (save / load / query positions);
- adding multi-room tasks: "find the bottle on the kitchen shelf".

5.3.3 Object Permanence and Semantic Memory

A scene graph updated across loop iterations would allow the agent to remember where objects were last seen and reason about persistence. This requires modest engineering: a Python dictionary mapping object labels to their last known 3-D positions in the base frame, updated at each successful detection.

5.3.4 Local VLM Deployment

Replacing the Gemini API with a locally deployed model (LLaVA 1.6, VILA, or InternVL) would eliminate the internet dependency and reduce latency. A GPU-equipped workstation connected via ROS would enable inference at <200 ms per query.

5.3.5 Adaptive Grasp Orientation

Extending the grasping pipeline to estimate the object's principal axis from the RANSAC cylinder fit and adapt the approach orientation accordingly would enable grasping of

non-cylindrical objects lying in arbitrary orientations — significantly expanding the range of manipulation tasks the system can handle.

5.3.6 Open-Vocabulary Detection

When GPU resources are available, replacing YOLO11n with an open-vocabulary detector such as Grounding DINO [23] or YOLOE [32] would remove the restriction to COCO-80 classes and enable fully open-ended task specifications: *"bring me the red mug"* rather than *"bring me the bottle"*.

5.4 Final Remarks

The demo that motivated this work — a user saying *"Hello TIAGo, I am thirsty, can you do something about that?"* and the robot searching, grasping a bottle, finding the user, and handing it over — is a small but concrete step toward robots that are genuinely useful as everyday companions.

What makes it noteworthy is not the individual components, each of which has been demonstrated in isolation many times before, but the *integration*: a physical robot, in a real room, reasoning about what to do with a cloud brain, and executing it with classical robotic control. The gap between language understanding and physical action is closed — not by end-to-end learning, but by a carefully engineered bridge between the two worlds.

As VLMs become faster, cheaper, and more capable, the reasoning quality will improve for free. The architecture built here is designed to benefit directly from those improvements without requiring a redesign.

Bibliography

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 1877–1901, 2020.

[3] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[4] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[5] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[6] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 24824–24837, 2022.

[7] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[8] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Inner monologue: Embodied reasoning through planning with language models," in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, 2022.

[9] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.

[10] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, R. Ring, E. Rutherford, S. Cabi, T. Han, Z. Gong, S. Samangooei, M. Monteiro, J. L. Menick, S. Borgeaud, A. Brock, A. Nematzadeh, S. Sharifzadeh, M. Binkowski, R. Barreira, O. Vinyals, A. Zisserman, and K. Simonyan, "Flamingo: A visual language model for few-shot learning," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 23716–23736, 2022.

[11] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[12] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Reymann, M. S. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, S. Singh, D. Sinha, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, A. Zeng, and T. Zhang, "Do as I can, not as I say: Grounding language in robotic affordances," in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, 2022.

[13] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Reymann, M. S. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, S. Singh, D. Sinha, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, A. Zeng, and T. Zhang, "RT-1: Robotics transformer for real-world control at scale," in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

[14] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," *arXiv preprint arXiv:2307.15818*, 2023.

[15] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, W. Huang, A. Zeng, K. Hausman, M. Toussaint, K. Greff, A. Zeng,

I. Mordatch, and P. Florence, "PaLM-E: An embodied multimodal language model," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[16] J. Lin, H. Yin, W. Shi, Y.-L. Chen, A. Chadha, H. Shi, and S. Han, "VILA: On preference alignment for vision-language models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[17] M. J. Kim, K. Pertsch, S. Karamcheti, T. Mukhopadhyay, A. Nair, S. Balakrishna, G. Berseth, D. Dass, C. Finn, A. Garg, K. Hausman, Y. Lassooij-Farahani, S. Levine, A. Loquercio, C. Lu, T. Nakamura, K. Rao, D. Sadigh, N. Silberman, *et al.*, "OpenVLA: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.

[18] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, "π_0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.

[19] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.

[20] Ultralytics, "YOLO11: The latest Ultralytics YOLO model." https://github.com/ultralytics/ultralytics, 2024. GitHub repository.

[21] T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, "YOLO-World: Real-time open-vocabulary object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[22] M. Minderer, A. Gritsenko, A. Stone, M. Neumann, D. Weissenborn, A. Dosovitskiy, A. Mahendran, A. Arnab, M. Dehghani, Z. Shen, X. Wang, X. Zhai, T. Kipf, and N. Houlsby, "Simple open-vocabulary object detection with vision transformers," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022.

[23] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024.

[24] R. Schnabel, R. Wahl, and R. Klein, "Efficient RANSAC for point-cloud shape detection," *Computer Graphics Forum*, vol. 26, no. 2, pp. 214–226, 2007.

[25] Y. Xiang, T. Schmidt, V. Narayanan, and D. Fox, "PoseCNN: A convolutional neural network for 6d object pose estimation in cluttered scenes," in *Proceedings of Robotics: Science and Systems (RSS)*, 2018.

[26] B. Wen, W. Yang, J. Kautz, and S. Birchfield, "FoundationPose: Unified 6d pose estimation and tracking of novel objects," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[27] J. Mahler, J. Liang, S. Niyaz, F. Pokorny, R. Calandra, A. Luo, and P. Abbeel, "Dex-net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics," in *Proceedings of Robotics: Science and Systems (RSS)*, 2017.

[28] H.-S. Fang, C. Wang, M. Gou, and C. Lu, "GraspNet-1Billion: A large-scale benchmark for general object grasping," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[29] M. Sundermeyer, A. Mousavian, R. Triebel, and D. Fox, "Contact-GraspNet: Efficient 6-dof grasp generation in cluttered scenes," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2021.

[30] I. A. Sucan and S. Chitta, "MoveIt!," in *IEEE Robotics & Automation Magazine*, 2013.

[31] T. Kunz and M. Stilman, "Time-optimal trajectory generation for path following with bounded acceleration and velocity," in *Proceedings of Robotics: Science and Systems (RSS)*, 2012.

[32] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOE: Real-time seeing anything," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.